TrackSense

An AI System for Predicting Allergen Cross Contact
Risk Propagation in Shared Kitchens through Computer Vision

A CREST Project Report

Prototype relative risk estimate. Not a biochemical allergen measurement. (physical_verification = false)

Table of Contents

1. Abstract

For this project I built TrackSense, a product that detects how allergen risk moves around a kitchen. It uses a single camera running YOLO to spot the objects involved, a contact event monitor to catch when those objects touch, and a risk model to estimate how contamination spreads from one object to the next. The result is shown live on a dashboard, so a user can see the risk as it develops.

The problem I wanted to solve is that in kitchens, schools, and catering companies, cross contamination spreads fast and without anyone noticing. An allergen can travel from one object to another in seconds, and by the time it reaches someone's food the original source is long gone. That is the gap TrackSense is built to fill.

I want to be clear from the start that TrackSense estimates risk, it does not measure allergens chemically. Every result is a relative risk estimate and the whole system is flagged with physical_verification = false. In this report I explain the research I did, the problem I was solving, my objectives, how I planned and scheduled the work, the method I followed, the datasets I used, and the results I got, including where the system worked well and where it did not.

Keywords: *Food safety, Allergens, Cross contact, YOLO, Computer Vision, Object Tracking, Risk Propagation, Local AI, FastAPI, React*

2. Wider Purpose

It matters to me that TrackSense is understood as more than just a product. Its main goal is not to sell a gadget, it is to protect people. Allergen cross contamination can seriously harm or even kill someone, and a great deal of that harm happens simply because the risk was invisible at the moment it mattered. If a tool like this can make that risk visible in time, then its real value is measured in harm avoided and lives protected, not in units sold.

Because of that, I see a strong non profit side to this work. The same system that could run in a commercial catering kitchen could just as easily run in a school canteen, a community kitchen, or a family home, where there is no budget for expensive safety equipment but where the people at risk are often children. Keeping the system local and low cost is a deliberate part of that purpose: it means the benefit can reach the people who need it most, rather than only those who can pay.

There is also an educational purpose. Part of what I want TrackSense to do is help people actually identify allergens and understand how they move, so that even the act of using it raises awareness. Many people simply do not realise how easily cross contact spreads, and a tool that shows it happening in front of them changes that perception. In that sense the wider purpose is twofold: to prevent harm directly, and to build a broader understanding of a danger that is usually invisible.

3. Background Research

I began by researching the problem from as many angles as I could. I read through a number of different websites on food allergy and cross contamination, and I spoke to parents, to experts, and to friends to understand what people actually deal with day to day. Talking directly to people who live with allergies was the most useful part, because it showed me how often a reaction is caused by something invisible rather than an obvious mistake.

3.1 What the research showed

The scale of the problem surprised me. More than 40 percent of children with food allergies have experienced a severe reaction such as anaphylaxis, which made it clear that this is not a rare edge case but something that affects a very large number of families. That was what convinced me the project was worth building.

I also learned that cross contact is different from having an allergen in a recipe. It is not about what is written on a label, it is about what physically touches what during preparation. A clean looking knife that was used for peanut butter can still carry enough residue to cause a reaction, and the danger is really about movement and sequence: the same objects can be safe or risky depending on the order in which they touch each other.

3.2 Existing tools and where they fall short

I looked at what already exists to help people avoid allergens, and almost everything works on a single item at a single moment. Portable sensors and testing devices chemically test one sample but only see the item in front of them. Ingredient and label scanning apps warn about known ingredients but say nothing about what happens during preparation. A few AI tools watch kitchens for hygiene, but they check individual rule breaking events rather than following how contamination actually spreads. The gap I found is that every tool answers "is the allergen here right now?", and none of them answer "where might the risk have travelled?". That is the gap TrackSense fills.

3.3 Technical research

On the technical side, I researched object detection and settled on YOLO, because it is fast enough to run on a live video feed and is well suited to spotting everyday objects. I also researched how to capture a camera stream, how to serve a live backend with FastAPI, and how to build the dashboard in React, which matched the design system I use across my other projects. Learning how these pieces fit together was a large part of the early work.

4. The Problem I Wanted to Solve

The problem I set out to solve is that cross contamination in a kitchen spreads fast and without anyone noticing. In a busy kitchen, a school canteen, or a catering company, an allergen can move from one object to another in seconds, and by the time it reaches someone's food the original source is long gone.

What makes this genuinely hard is the tracking. A real kitchen is full of moving objects, and it is extremely difficult to keep track of multiple pieces of cutlery and everything else at the same time, especially when they look similar, overlap, or move quickly in and out of view. Following every object and every contact between them, reliably and in real time, is the core challenge TrackSense has to deal with.

5. Aim and Objectives

Aim

The core idea behind TrackSense is simple: allergen cross contamination harms people, often seriously, and most of that harm is invisible until it is too late. My aim was to build something that could make this hidden risk visible, and in doing so help make cross contamination far more widely understood. In the longer term, the ambition is bigger than a single prototype. I want an approach like this to become widespread, in home kitchens, schools, and catering companies, so that it can genuinely reduce harm and, at its best, help save lives by catching risk before it reaches someone with an allergy.

Scope: what I set out to do

To keep that aim achievable, I defined a clear scope for what this project had to deliver. The task I set myself was to build a working, local system that watches a kitchen, detects the objects involved, recognises when they come into contact, estimates how allergen risk spreads between them, and shows that risk live in a way that is honest about its limits. Anything beyond that, such as chemical measurement or full commercial deployment, was deliberately left outside the scope of this build.

Objectives

Within that scope, I set myself a set of concrete objectives. I would have judged the project a success if:

- My system could detect the core kitchen objects reliably enough to run a live demonstration.
- It could recognise when objects came into contact and use that to estimate risk.
- That risk was shown live on a dashboard and was easy to read at a glance.
- Every output was honest about being a relative risk estimate, not a chemical measurement.
- The whole thing ran locally on a single laptop with one camera.

- The approach was simple and low cost enough that it could realistically be spread to real kitchens, schools, and catering settings.

6. Approaches I Considered

TrackSense was not my first idea. Before settling on it, I went through several different projects and approaches, and looking at why each of the others fell short is part of how I arrived at this one. I have set them out here because the process of ruling things out mattered as much as the final choice.

A posture correction project

My original project was a posture detection tool. The idea was to use a camera to check a person's posture and give them feedback. I scrapped it because in practice it was not effective enough, and it did not help a wide enough range of people to feel worth pursuing. It pushed me to look for a problem where the technology could make a clearer, more meaningful difference.

Counting people on trains

Another idea was to track how many people were on trains, using cameras at stations. This one fell short for a practical reason: at somewhere like a Mumbai train station, I could not realistically run a model powerful enough to do the job reliably in that environment. The gap between what the idea needed and what could actually be deployed on site was too large.

Stopping phishing emails for older people

I also spent time on an idea for protecting older people from phishing emails, which is a real and serious problem for that group. It was a promising direction, but it did not develop into something I felt I could build well enough within the scope of this project.

An AI therapy app

At one point I almost built an AI therapy app. I stepped away from it because it did not work effectively, and because a tool in that space carries a level of responsibility that I did not think a prototype could safely meet.

Final approach

Each of these attempts taught me something about what I was actually looking for: a problem that was real and affected a lot of people, that was technically achievable with the tools I had, and that could be demonstrated honestly. TrackSense fit all three, which is why I committed to it.

The final approach I settled on was a single local system that ties together three parts. A camera running YOLO detects the kitchen objects, a contact event monitor watches for when those objects touch, and a risk model estimates how contamination is likely to spread from one object to the next. The result is shown live on a dashboard so the risk is visible as it develops. Everything runs locally on one laptop, which keeps it cheap to deploy and means no kitchen footage has to leave the room.

I chose this over the alternatives because it was the only design where I could actually show risk moving, rather than just detecting a single object at a single moment. It was realistic to build in the time I had, it addressed a problem that genuinely affects a large number of people, and it could be presented honestly as a relative risk estimate rather than something it was not. Those were exactly the qualities the earlier ideas were missing, and that comparison is what gave me the confidence to build TrackSense properly.

7. How I Planned the Project

My planning was built around getting advice before committing to anything. I spoke to a lot of different people, debated ideas back and forth with my mentors, and consulted several experts to pressure test my approach. Those conversations shaped what I chose to build and, just as importantly, what I chose to leave out.

From that input I made a few key planning decisions early. I locked a fixed set of kitchen objects to detect so that nothing downstream would keep shifting, choosing eight classes that covered exactly the nut transfer scenario and no more, and I deliberately dropped a "counter" class after finding that wood grain surfaces caused false detections. Locking this early meant my data preparation, detection, contact logic, and dashboard could all rely on a stable set of class IDs.

I then planned the build in clear stages: prepare the data, get the live pipeline running, build the dashboard, then test on real objects and stage a demonstration. I also decided from the start to treat improving the model as a separate track, so that experimenting with it could never break the working system I needed to show.

The eight class schema I locked:

ID	Object	Role in the risk chain
0	nut_butter_jar	Primary allergen source
1	whole_nuts	Secondary allergen source
2	hand	Mobile carrier between objects
3	cutlery	Transfer vector (jar to bread)
4	chopping_board	Shared surface / transfer point
5	plate	Downstream receiving object
6	bowl	Downstream receiving object
7	bread	Final consumed item

8. Project Timeline (Gantt Chart)

My project ran in two very different phases. For the first month I was mostly researching and talking about the idea, speaking with experts, debating the approach with my mentors, and refining what I actually wanted to build. Then, in the final week before the demonstration,

everything shifted into an intense build phase. In those last seven days I did the bulk of the practical work, staying up late to finish, still researching, still asking questions, and putting in a lot of hours to get a working system ready in time. The chart below shows that shape across five weeks.

Stage	W1	W2	W3	W4	W5
Research & talking					
Experts & mentors					
Refining the idea					
Building the system					
Testing & final push					
Demo & write up					

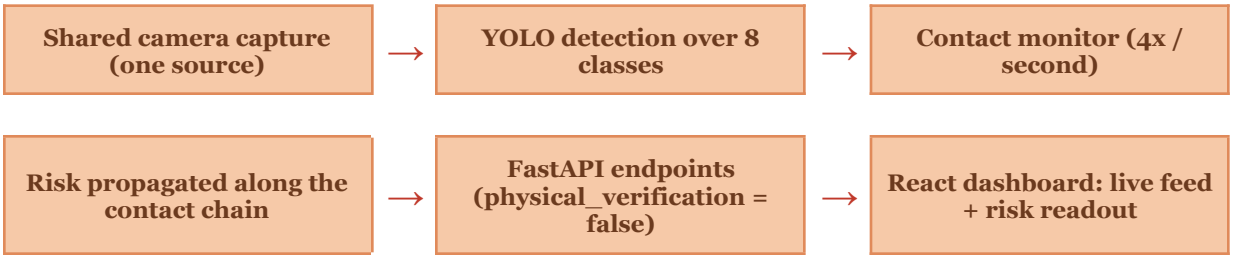
Filled bars show active work. W1 to W5 are project weeks; the bulk of the build happened in the final week.

The table below breaks the project down by the hours I spent on each stage. In total I put roughly 82 hours into TrackSense.

Stage	What it involved	Hours
Research	Reading, primary research, understanding the problem	20
Experts & mentors	Speaking with experts and debating the idea with mentors	15
Refining the idea	Narrowing the concept and locking the approach	7
Building the system	Detection, contact logic, risk model, dashboard, API	20
Testing & final push	Testing on real objects and the final build sprint	10
Demo & write up	Preparing the demonstration and writing this report	10
Total		**82**

9. Methodology

Here I explain how I actually built the system, step by step, in the order the data flows through it. The overall flow is shown below.



A single shared camera

I built the whole system around one camera and one continuous detection loop. I made the camera capture shared, so a single frame source feeds both the live detector and the contact monitor. Early on I learned that opening the camera more than once causes the components to fight over the device, so sharing one capture was an important design choice.

Detecting objects

Each frame from the camera goes through my YOLO model, which returns the objects it can see as boxes with class labels. I send these results to two places at once: to the dashboard so the user can see what is detected, and to the contact monitor so the system can reason about what is touching what.

Tracking contact and spreading risk

A monitor checks the current detections four times a second and looks for objects whose boxes overlap. When two objects are in contact and one of them already carries risk, I pass that risk to the other, moving it one step further along the chain. This is how I model risk travelling from the nut butter jar, onto the cutlery, onto the bread, and onto the plate. The value I pass along is always a relative estimate of how likely exposure is, never a measured amount of allergen.

Serving it and showing it

I wrote a FastAPI backend with a small set of endpoints covering live detection, the shared camera frame, the contact monitor state, and the current risk readout. Across all of these I enforced physical_verification = false, so no part of the system can ever report a result as a confirmed physical measurement. On top of the API, my React and Tailwind dashboard shows the live camera view with detection overlays and keeps the current risk level prominent and easy to read.

Testing

I backed the pipeline with an automated test suite so that when I changed one part of the system, I could check the detection loop, the endpoints, and the risk logic without re testing everything by hand. This let me move quickly without breaking things I had already got working.

Datasets I used

No single public dataset matched my eight class schema, so I had to build my training data by collapsing several external datasets down into my own classes. This was one of the more time consuming parts of the project.

For the nut classes, I took many different nut subcategories from external datasets and mapped them all down to my single whole_nuts class. I built the nut_butter_jar and whole_nuts sets to a validated one to one balance and standardised every class ID to match my canonical schema, so the model would not be confused by mismatched labels. I also downloaded a set of extra datasets in COCO format as insurance for improving the weaker classes later, covering nuts,

peanuts, kitchen utensils, general kitchen objects, and bread, and kept these as a separate track so they could not interfere with my working model.

Dataset	Source	What I used it for
Nut butter / whole nuts (own set)	Compiled, validated 1:1	Core training for classes 0 and 1
Nut classification, Peanuts	Roboflow (COCO)	Insurance for whole_nuts retrain
Kitchen Utensils Recognition	Roboflow (COCO)	Insurance for cutlery
kitchen object detection	Roboflow (COCO)	Counter surfaces to fix board false positives
Bread Detection	Roboflow (COCO)	Insurance for bread

Dataset split. Because the risk model needs its own training data, I generated a set of contact event scenarios and split them by scenario (not by row) so that no scenario could leak between training and testing. The split below is the real breakdown from my training run: a 70 / 15 / 15 division across 5,000 scenarios, giving 24,570 individual contact events in total.

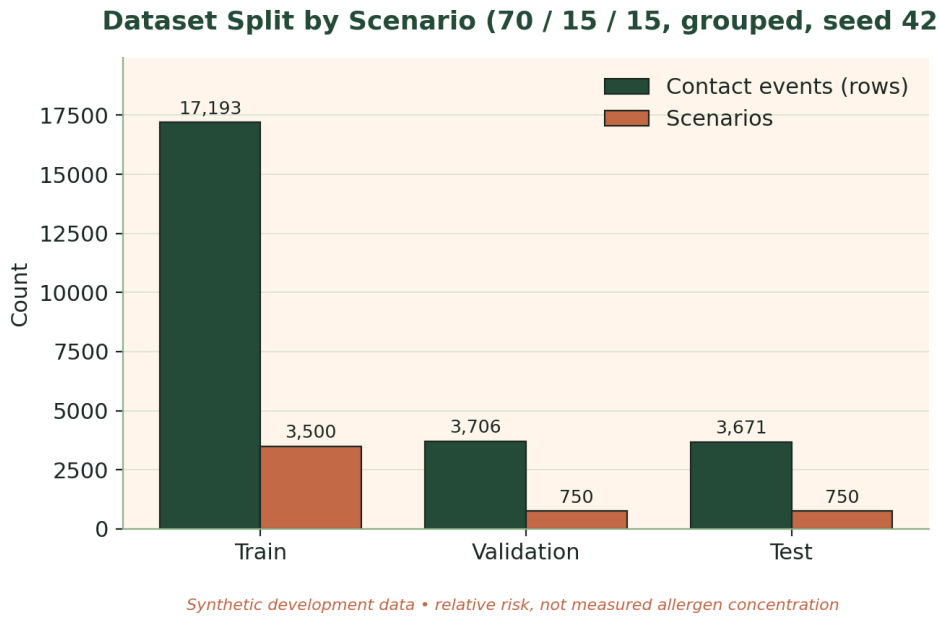


Figure 1: Dataset split by scenario for the risk model (real training run figures).

10. Results

In this section I set out what my system takes as input, how well it performed, and where it fell short. I have tried to be honest about the weak points, because a food safety tool that hides its own unreliability is worse than useless.

Inputs

The input to my system is a live video feed from one camera pointed at a work surface. Each frame is a single image that the detector processes. The system does not need any other input

from the user: it reads the scene, decides what objects are present, works out what is touching what, and updates the risk estimate on its own.

What worked

The live pipeline worked end to end. I had continuous detection running off a single shared camera, the full set of API endpoints responding, the contact monitor polling four times a second, the honesty constraint enforced everywhere, and the automated test suite passing. The core idea, risk spreading along a chain of contacts, worked as I designed it.

Detection accuracy by object

The honest limitation was not the pipeline but how reliably the detector recognised real physical objects. Some classes performed much better than others when I tested them on camera:

Object	Detection reliability	Notes
nut_butter_jar	Strong	Safe to build a demo around
bread	Strong	Reliable in the scene
cutlery (metal)	Good	Detects dependably
cutlery (plastic)	Borderline	Much weaker than metal
chopping_board	Conditional	False positives on wood grain
whole_nuts	Weak	Near zero at low confidence
bowl	Weak	Near zero at low confidence

Note: the reliability column above is based on my own on camera testing. I have left the exact numerical metrics (such as mAP and per class recall) to be filled in from my training logs, so that the figures in this report are the real ones rather than estimates.

How I responded to the weak classes

Rather than pretend the weak classes worked, I staged my demonstration around the strong ones. I used the nut butter jar, metal cutlery, bread, and a plate on a plain, non wood surface. That single choice avoids every known weak point at once: the unreliable classes are simply not in the scene, metal cutlery replaces plastic, and the plain surface stops the wood grain false positives. I also kept a headless recording of the full pipeline as a backup, so the demonstration would hold up even if the live setup misbehaved.

What I would do next

My main planned improvement is to retrain the detector on the weak classes, whole_nuts, bowl, and plastic cutlery, using the extra datasets I had already gathered. I also want to add proper counter surface classes to cleanly separate a chopping board from a work surface and remove the wood grain problem for good. I deliberately left this retrain until after the live demonstration, so I would never disturb the working model while it was the thing I needed to show.

11. Reflections and Improvements

Now that the project is finished, there are a number of things I would do differently, and a few honest reflections on how it went.

What I would rethink

My main reflection is that I could have spent more time thinking through the project before diving into the build. Because so much of the practical work was compressed into the final week, some decisions were made under time pressure that I would rather have planned out more carefully. Giving myself more room to think early on would have made the later stages smoother and probably produced a stronger result.

Improving model accuracy

The clearest technical improvement is the model itself. I believe I could improve the detector's accuracy simply by training it more, on more data and for longer, since the weakest classes suffered most from having too few examples. Retraining on the extra datasets I had already gathered, and giving the model more epochs to learn from, is the most direct way to lift the classes that currently underperform, such as whole_nuts, bowl, and plastic cutlery.

Wider lessons

More broadly, this project taught me that the hard part of a physical computer vision system is not the architecture, it is getting the detector to reliably see small, plain, or variable objects in a real scene. Building a correct pipeline was achievable in the time I had, but making the model trustworthy on every object is a longer road. It also taught me the value of scoping tightly and being honest about limitations, rather than overstating what a prototype can do. Those are lessons I will carry into whatever I build next.

12. Ethical and Safety Considerations

Because TrackSense uses a camera in a kitchen and deals with people's safety, I thought carefully about the ethical and safety issues from the start.

Privacy and data

The most important point is that the camera is not used to collect any data. TrackSense reads the live video only to work out what objects are present and what is touching what, and it does not save, store, or record any images or footage of the people in the kitchen. Nothing is kept and nothing is sent anywhere. The whole system runs locally on a single laptop, so the video never has to leave the room. This was a deliberate choice: a tool meant to protect people should not create a new privacy risk of its own.

Not giving false confidence

The second safety issue is honesty. Because this concerns food safety risk for people with serious allergies, the worst thing the system could do is make someone feel safe when they are not. That is why every output is presented as a relative risk estimate and the whole system is flagged with

physical_verification = false. TrackSense estimates risk, it does not chemically confirm the presence or absence of an allergen, and it is not a replacement for allergen testing, careful hygiene, or a person's own judgement. Being clear about that limit is part of using the tool responsibly.

Responsible use

Taken together, these choices are meant to make TrackSense safe to use: it protects the privacy of the people it watches, and it is honest about what it can and cannot tell you. It is designed to support better awareness and earlier action, not to be relied on as the final word on whether food is safe.

13. Conclusion

Looking back, I think TrackSense succeeded at the thing I set out to do: it makes an invisible process, the movement of allergen risk through a kitchen, visible in real time, while staying honest that it estimates rather than measures. I met my objectives for the pipeline, the contact tracking, the risk logic, and the dashboard, and I was honest about where the detector still needs work.

Most importantly, the project reinforced why it matters. Cross contamination is a real danger that harms people precisely because it is invisible, and a tool that makes it visible, kept cheap and local enough to reach the kitchens that need it, could do genuine good. That is the direction I would want to take TrackSense next.

14. Bibliography

The sources below informed my research. Each title links to the original paper.

Food allergy and cross contact

Gupta et al. 2019, US adult food allergy prevalence 10.8% of US adults estimated food allergic nationally.

Gupta et al. 2018, US childhood food allergy impact 7.6% of children affected; 42% ED visit rate.

Taylor & Baumert 2010, cross contamination review Frequency of cross contact causing reactions is unknown.

Furlong et al. 2001, nut reactions in restaurants 22% of reactions from shared cooking supplies.

Perry et al. 2004, peanut allergen in environment Cleaning method determines whether surface residue persists.

Versluis et al. 2015, unexpected allergic reactions review Eating outside home is a leading reaction context.

Ortiz et al. 2018, allergens on canteen surfaces Gluten found on 57% of cleaned school surfaces.

Galan Malo et al. 2019, reducing surface contamination Study reducing allergen residue on canteen surfaces.

Konstantinou et al. 2025, allergies in dining establishments Reviews hidden allergens, prep cross contamination, staff communication.

Existing detection approaches and the gap

Taylor et al. 2018, handheld gluten detector evaluation Nima sensor reliable above 20ppm, misses matrices.

Maric & Scherf 2021, portable gluten sensor reliability Inconsistent detection in the low concentration band.

Ince et al. 2023, lateral flow assays review Rapid on site screening for food allergens, contaminants.

Chen et al. 2025, deep learning for hygiene Closest published relative: vision for kitchen safety.

Revelou et al. 2025, machine learning in food safety Machine learning applied to HACCP monitoring tasks.

Taylor et al. 2014, VITAL reference doses report Establishes reference doses for allergenic food residues.

Remington et al. 2020, minimal eliciting dose distributions Eliciting dose distributions for fourteen priority allergens.

Houben et al. 2020, full eliciting dose values Population eliciting dose ranges for risk characterization.

Object detection in food and kitchen domains

Redmon et al. 2016, YOLO real time detection Foundational single stage real time object detection network.

Damen et al. 2018, EPIC KITCHENS egocentric dataset Large egocentric kitchen dataset with bounding boxes.

Damen et al. 2022, EPIC KITCHENS 100 rescaling Expanded egocentric kitchen benchmark, pipeline, challenges.

Small Object Detection 2025, comprehensive survey Surveys challenges and techniques for small objects.

Contact, interaction detection, and tracking

Shan et al. 2020, hands in contact at scale 100DOH dataset; predicts hand object contact state.

Narasimhaswamy et al. 2020, physical contact in wild ContactHands dataset; detects hands and physical contact.

Ji et al. 2020, Action Genome scene graphs Decomposes actions into graphs with contact relations.

Chao et al. 2018, HICO DET interaction detection Image level human object interaction detection, 600 categories.

Zhang et al. 2022, ByteTrack multi object tracking Tracks by associating every detection box, including low confidence.

Darkhalil et al. 2022, EPIC KITCHENS VISOR benchmark Pixel level hand and active object segmentation relations.

HOI Detection 2020, quick survey of methods Survey entry point for human object interaction detection.

Risk and contamination propagation modeling

Iulietto & Evers 2024, kitchen QMRA model Models chained surface to surface transfer for risk assessment.

Iulietto & Evers 2020, cross contamination magnitude estimation Estimates fraction of contamination ultimately ingested.

Remington et al. 2020, shared restaurant equipment risk Simulated peanut exposure through shared kitchen equipment.

Spanjersberg et al. 2007, probabilistic allergen risk model Underlying probabilistic risk assessment methodology for allergens.

Wang et al. 2021, surface touch network spread Network structure determines contamination spread across surfaces.

Lei et al. 2017, surface contamination network growth Contaminated surfaces grow logistically, driving disease spread.

Kraay et al. 2018, fomite transmission across pathogens Compares fomite mediated transmission across three viral pathogens.

Zhao et al. 2012, fomite influenza transmission model EITS model comparing surface mediated transmission pathways.